```
Start coding or generate with AI.
```

PRACTICAL NO 03

Name : Pranjal somatkar roll no. : B48

AIM: Data Wragling With Pandas: Manipulate and transform data to suit the analysis needs.

Data Wrangling with Pandas (Theory)

1. What is Data Wrangling? Explain the difference between Data Wrangling and Data Preprocessing.

Data Wrangling:

Data wrangling (also called data munging) is the process of **cleaning, transforming, and organizing raw data** into a structured and usable format for analysis. In real-world scenarios, raw data is often messy, incomplete, inconsistent, or unstructured. Data wrangling helps convert this raw data into a form suitable for analysis or modeling.

It includes tasks such as:

- Removing duplicate values
- Handling missing data
- Converting data types
- Filtering and sorting data
- Merging datasets
- Reshaping data

Using libraries like **pandas**, data wrangling becomes efficient and easy to perform.

Difference between Data Wrangling and Data Preprocessing:

Aspect	Data Wrangling	Data Preprocessing
Definition	Cleaning and transforming raw data into usable format	Preparing data specifically for machine learning models
Focus	Data organization and cleaning	Data optimization for algorithms
Scope	Broad (includes cleaning, merging, restructuring)	Narrow (focus on model input preparation)
Techniques	Filtering, merging, reshaping, handling missing values	Scaling, normalization, encoding categorical variables
Usage	Used in data analysis and exploration	Used in machine learning pipelines

2. What is the importance of Exploratory Data Analysis (EDA) in Pandas?

Exploratory Data Analysis (EDA) is a crucial step in data analysis where we **analyze and summarize datasets** using statistical and visualization techniques. In pandas, EDA helps in understanding the data before applying any advanced analysis or machine learning.

Importance of EDA:

-
-

1. Understanding Data Structure

EDA helps to understand:

- Number of rows and columns
- Data types of each column
- Overall dataset structure

Functions used:

- `df.shape`
- `df.info()`

---

2. Detecting Missing Values

EDA helps identify missing or null values which may affect analysis.

Functions:

- `isnull()`
- `sum()`

---

3. Identifying Patterns and Trends

By analyzing data, we can:

- Identify relationships between variables
- Detect trends and patterns

---

4. Finding Outliers

Outliers can distort results. EDA helps detect unusual values.

---

5. Summary Statistics

Provides quick insights into the dataset.

Functions:

- `df.describe()`

---

6. Data Cleaning Decisions

EDA helps decide:

- Which columns to remove
- How to handle missing values
- Whether transformation is needed

3. What are the main differences between Series, DataFrame, and Panel?

In pandas, data is structured into different formats depending on complexity:

---

1. Series

A Series is a **one-dimensional labeled array** capable of holding data of any type.

Features:

- Like a single column or list

Has index and values
Can store integers, strings, floats

Example:

```
import pandas as pd
s = pd.Series([10, 20, 30])
```

---

2. DataFrame

A DataFrame is a **two-dimensional labeled data structure** with rows and columns.

Features:

- Like a table (Excel sheet)
- Multiple columns with different data types
- Most commonly used pandas structure

Example:

```
data = {'Name': ['A', 'B'], 'Marks': [90, 85]}
df = pd.DataFrame(data)
```

---

3. Panel

A Panel is a **three-dimensional data structure** (now deprecated in pandas).

Features:

- Used to store 3D data
- Similar to multiple DataFrames combined
- Replaced by multi-index DataFrames

Difference Table:

Feature	Series	DataFrame	Panel
Dimension	1D	2D	3D
Structure	Single column	Table (rows & columns)	Collection of DataFrames
Usage	Simple data	Most common structure	Rare (deprecated)
Example	List-like	Excel-like	3D data storage

4. Explain the use of:

a. `isnull()`

The `isnull()` function in pandas is used to **detect missing or null values** in a dataset. It returns a DataFrame (or Series) of the same shape where:

- `True` represents missing values
- `False` represents non-missing values

Why it is useful:

- Helps identify incomplete data

- Used before cleaning or preprocessing
- Helps decide how to handle missing values

Example:

```
df.isnull()
```

---

b. `notnull()`

The `notnull()` function is the opposite of `isnull()`. It is used to **identify non-missing values**.

- `True` → value is present
- `False` → value is missing

Why it is useful:

- Helps filter valid data
- Useful when selecting only non-null rows

Example:

```
df.notnull()
```

---

c. `dropna()`

The `dropna()` function is used to **remove missing values** from the dataset.

Types of usage:

- Remove rows with missing values
- Remove columns with missing values

Parameters:

- `axis=0` → remove rows
- `axis=1` → remove columns
- `how='any'` → drop if any value is missing
- `how='all'` → drop if all values are missing

Why it is useful:

- Cleans dataset
- Improves data quality

Example:

```
df.dropna()
```

---

d. `fillna()`

The `fillna()` function is used to **replace missing values** instead of removing them.

Ways to fill values:

Fill with constant value (e.g., 0)
Fill with mean, median, or mode

- Forward fill (`ffill`)
- Backward fill (`bfill`)

Why it is useful:

- Prevents data loss
- Maintains dataset size

Example:

```
df.fillna(0)
```

---

5. What is Column Transformation and why is it important in data analysis?

Column Transformation:

Column transformation refers to the process of **modifying, converting, or creating new columns** in a dataset to make it more suitable for analysis.

It involves changing the format, values, or structure of columns.

Types of Column Transformation:

1. Changing Data Types

Converting columns into appropriate formats:

- String ⟶ Integer
- Float ⟶ Integer

Example:

```
df['Age'] = df['Age'].astype(int)
```

---

2. Creating New Columns

New columns can be created based on existing data.

Example:

```
df['Total'] = df['Math'] + df['Science']
```

---

3. Applying Functions

Using functions to modify column values.

Example:

```
df['Marks'] = df['Marks'].apply(lambda x: x + 5)
```

-

4. Encoding Categorical Data

Converting text data into numeric format.

Example:

```
pd.get_dummies(df['Gender'])
```

---

5. Normalization / Scaling

Adjusting values to a common range.

Importance of Column Transformation:

1. Improves Data Quality

Transforms raw data into a clean and usable format.

2. Enhances Analysis

Makes data easier to analyze and interpret.

3. Required for Machine Learning

Many algorithms require numeric and well-structured data.

4. Feature Engineering

Helps create meaningful features that improve model performance.

5. Handles Inconsistencies

Fixes errors such as incorrect formats or mixed data types.

Load Dataset

```
import pandas as pd


# Load dataset
df = pd.read_csv("/Iris[1].csv")

# Display first rows
print(df.head())
```

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa
```

BASIC TASKS

Task 1: Select a single column

```
6
```

```
import pandas as pd

df = pd.read_csv("Iris[1].csv")

# Select single column
column = df['Id']
print(column.head())
```

```
0    1
1    2
2    3
3    4
4    5
Name: Id, dtype: int64
```

Task 2: Add a new column with constant values

```
import pandas as pd

df = pd.read_csv("Iris[1].csv")

# Add constant column
df['constant_col'] = 1
print(df.head())
```

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species  \
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa

   constant_col
0             1
1             1
2             1
3             1
4             1
```

Task 3: Sort DataFrame by one column

```
import pandas as pd

df = pd.read_csv("Iris[1].csv")

# Sort by Id
sorted_df = df.sort_values(by='Id')
print(sorted_df.head())
```

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa
```

Task 4: Rename DataFrame columns

```
import pandas as pd

df = pd.read_csv("Iris[1].csv")
```

```
# Rename column
df = df.rename(columns={'Id': 'ID_New'})
print(df.head())
```

```
   ID_New  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  \
0       1            5.1           3.5            1.4           0.2
1       2            4.9           3.0            1.4           0.2
2       3            4.7           3.2            1.3           0.2
3       4            4.6           3.1            1.5           0.2
4       5            5.0           3.6            1.4           0.2

       Species
0  Iris-setosa
1  Iris-setosa
2  Iris-setosa
3  Iris-setosa
4  Iris-setosa
```

Task 5: Display summary statistics

```
import pandas as pd

df = pd.read_csv("Iris[1].csv")

# Summary statistics
print(df.describe())
```

```
               Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
count  150.000000     150.000000    150.000000     150.000000    150.000000
mean    75.500000       5.843333      3.054000       3.758667      1.198667
std     43.445368       0.828066      0.433594       1.764420      0.763161
min      1.000000       4.300000      2.000000       1.000000      0.100000
25%     38.250000       5.100000      2.800000       1.600000      0.300000
50%     75.500000       5.800000      3.000000       4.350000      1.300000
75%    112.750000       6.400000      3.300000       5.100000      1.800000
max    150.000000       7.900000      4.400000       6.900000      2.500000
```

MODERATE TASKS

Task 6: Filter rows using multiple conditions

```
import pandas as pd

df = pd.read_csv("Iris[1].csv")

# Filter condition
filtered_df = df[(df['SepalLengthCm'] > 5) & (df['PetalLengthCm'] > 1.5)]
print(filtered_df.head())
```

```
    Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
5    6            5.4           3.9            1.7           0.4  Iris-setosa
18  19            5.7           3.8            1.7           0.3  Iris-setosa
20  21            5.4           3.4            1.7           0.2  Iris-setosa
23  24            5.1           3.3            1.7           0.5  Iris-setosa
44  45            5.1           3.8            1.9           0.4  Iris-setosa
```

Task 7: Group data and calculate mean

```
import pandas as pd
```

```
df = pd.read_csv("Iris[1].csv")

# Group by Species
grouped = df.groupby('Species').mean(numeric_only=True)
print(grouped)
```

```
                   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  \
Species
Iris-setosa      25.5          5.006         3.418          1.464
Iris-versicolor  75.5          5.936         2.770          4.260
Iris-virginica  125.5          6.588         2.974          5.552

                PetalWidthCm
Species
Iris-setosa            0.244
Iris-versicolor        1.326
Iris-virginica         2.026
```

Task 8: Merge two DataFrames

```
import pandas as pd

df1 = pd.read_csv("Iris[1].csv")
df2 = pd.read_csv("Iris[1].csv")

# Merge on Id
merged_df = pd.merge(df1, df2, on='Id')
print(merged_df.head())
```

```
   Id  SepalLengthCm_x  SepalWidthCm_x  PetalLengthCm_x  PetalWidthCm_x  \
0   1              5.1             3.5              1.4             0.2
1   2              4.9             3.0              1.4             0.2
2   3              4.7             3.2              1.3             0.2
3   4              4.6             3.1              1.5             0.2
4   5              5.0             3.6              1.4             0.2

     Species_x  SepalLengthCm_y  SepalWidthCm_y  PetalLengthCm_y  \
0  Iris-setosa              5.1             3.5              1.4
1  Iris-setosa              4.9             3.0              1.4
2  Iris-setosa              4.7             3.2              1.3
3  Iris-setosa              4.6             3.1              1.5
4  Iris-setosa              5.0             3.6              1.4

   PetalWidthCm_y    Species_y
0             0.2  Iris-setosa
1             0.2  Iris-setosa
2             0.2  Iris-setosa
3             0.2  Iris-setosa
4             0.2  Iris-setosa
```

Task 9: Drop rows and columns

```
import pandas as pd

df = pd.read_csv("Iris[1].csv")

# Drop column
df = df.drop(columns=['SepalWidthCm'])

print(df.head())
```

```
   Id  SepalLengthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1            1.4           0.2  Iris-setosa
1   2            4.9            1.4           0.2  Iris-setosa
2   3            4.7            1.3           0.2  Iris-setosa
```

```
3   4            4.6            1.5           0.2  Iris-setosa
4   5            5.0            1.4           0.2  Iris-setosa
```

Task 10: Apply a function to a column

```
import pandas as pd

df = pd.read_csv("Iris[1].csv")

# Apply function
df['Id_Double'] = df['Id'].apply(lambda x: x * 2)

print(df.head())
```

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species  \
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa

   Id_Double
0          2
1          4
2          6
3          8
4         10
```

ADVANCED TASKS

Task 11: Pivot table

```
import pandas as pd

df = pd.read_csv("Iris[1].csv")

# Pivot table
pivot_table = pd.pivot_table(df, values='SepalLengthCm', index='Species')

print(pivot_table)
```

```
                SepalLengthCm
Species
Iris-setosa             5.006
Iris-versicolor         5.936
Iris-virginica          6.588
```

Task 12: Melt operation

```
import pandas as pd

df = pd.read_csv("Iris[1].csv")

# Melt
melted_df = pd.melt(df, id_vars=['Species'])

print(melted_df.head())
```

```
       Species variable  value
0  Iris-setosa       Id    1.0
```

0

```
1  Iris-setosa       Id    2.0
2  Iris-setosa       Id    3.0
3  Iris-setosa       Id    4.0
4  Iris-setosa       Id    5.0
```

Task 13: Multi-level indexing

```
import pandas as pd

df = pd.read_csv("Iris[1].csv")


# Multi-level index
multi_index_df = df.set_index(['Species', 'Id'])

print(multi_index_df.head())
```

```
                SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
Species     Id
Iris-setosa 1             5.1           3.5            1.4           0.2
            2             4.9           3.0            1.4           0.2
            3             4.7           3.2            1.3           0.2
            4             4.6           3.1            1.5           0.2
            5             5.0           3.6            1.4           0.2
```

Task 14: Custom aggregation function

```
import pandas as pd

df = pd.read_csv("Iris[1].csv")

# Custom aggregation
custom_agg = df.groupby('Species').agg(
    lambda x: x.max() - x.min() if x.dtype != 'object' else None
)

print(custom_agg)
```

```
                 Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
Species
Iris-setosa      49            1.5           2.1            0.9           0.5
Iris-versicolor  49            2.1           1.4            2.1           0.8
Iris-virginica   49            3.0           1.6            2.4           1.1
```

Task 15: Time series based data wrangling

```
import pandas as pd

df = pd.read_csv("Iris[1].csv")

# Create date column
df['Date'] = pd.date_range(start='2020-01-01', periods=len(df), freq='D')

# Time series resampling
time_series = df.set_index('Date').resample('ME').mean(numeric_only=True)

print(time_series.head())
```

```
               Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
Date
2020-01-31   16.0       5.019355      3.438710       1.477419      0.245161
2020-02-29   46.0       5.368966      3.206897       2.451724      0.634483
```

```
2020-03-31   76.0       5.964516      2.745161       4.303226      1.335484
2020-04-30  106.5       6.300000      2.876667       5.156667      1.810000
2020-05-31  136.0       6.596552      3.003448       5.475862      2.003448
```

Conclusion

In conclusion, data wrangling is a fundamental and essential process in the field of data analysis, as it focuses on converting raw, unstructured, and messy data into a clean, organized, and meaningful format. Real-world data is often incomplete, inconsistent, and noisy, which makes data wrangling a necessary step to ensure that the data becomes reliable and suitable for further analysis. It includes various operations such as handling missing values, removing duplicates, correcting errors, and transforming data into appropriate formats.

Along with data wrangling, Exploratory Data Analysis (EDA) plays a vital role in understanding the dataset in depth. EDA allows analysts to explore the structure of the data, identify patterns and trends, detect outliers, and gain useful insights. It also helps in making informed decisions regarding data cleaning and transformation. Without performing proper EDA, it becomes difficult to understand the nature of the data, which may lead to incorrect analysis and poor results.

The use of pandas data structures such as Series and DataFrame provides a powerful and flexible way to store, manage, and manipulate data efficiently. These structures make it easier to perform various operations like filtering, grouping, and aggregation, which are essential for data analysis tasks. Understanding these data structures is important for working effectively with datasets.

Handling missing data using functions like `isnull()`, `notnull()`, `dropna()`, and `fillna()` is another critical aspect of data wrangling. Missing values can significantly affect the quality and accuracy of analysis if not handled properly. These functions help in identifying, removing, or replacing missing values, thereby improving the consistency and reliability of the dataset.

Moreover, column transformation is an important technique used to modify and enhance data according to analysis requirements. It includes operations such as changing data types, creating new columns, applying functions, and encoding categorical data. These transformations help in improving data quality and preparing it for advanced analysis and machine learning models.

Overall, all these concepts—data wrangling, EDA, data structures, handling missing values, and column transformation—are interconnected and play a significant role in the data analysis pipeline. They ensure that the data is accurate, consistent, and meaningful, which ultimately leads to better insights, improved decision-making, and more reliable outcomes. Therefore, mastering these techniques is essential for anyone working in data analysis, data science, or related fields.